

RisiKo! un mockup

Azhar Hamail, Morri Simone, Zangheri Marco

27 settembre 2026

Sommario

Questo progetto porta a riprodurre il gioco del RisiKo!, o come viene conosciuto al di fuori dell'italia Risk, in modo da simularne, sebbene in maniera semplificata, comportamenti in modo da avere un'esperienza di gioco fluida ed equiparabile a quella del gioco da tavolo con la capacità di poter giocare in assenza di altri giocatori contro il computer.

Indice

1	Analisi	3
1.1	Descrizione e requisiti	3
1.1.1	Requisiti funzionali	4
1.2	Modello del Dominio	4
2	Design	6
2.1	Architettura	6
2.1.1	Carte e fasi del turno	7
2.2	Design dettagliato	8
2.2.1	Far produrre allo stesso tipo di entità risultati differenti. Morri Simone	8
2.2.2	Comunicazione per i Player. Morri Simone	9
2.2.3	Creare i Giocatori. Morri Simone	10
2.2.4	Zangheri Marco. Costruire la mappa e collegarla ai giocatori	11
2.2.5	Zangheri Marco. Dadi sostituibili e combattimento	12
2.2.6	Zangheri Marco. La mappa a schermo	13
2.2.7	Zangheri Marco. Cosa vuol dire un click	14
2.2.8	Zangheri Marco. Il flusso della partita	15
2.2.9	Hamail Azhar. Costruire due tipi di mazzi	15
2.2.10	Hamail Azhar. Condizioni di vittoria	16
2.2.11	Hamail Azhar. Armate bonus in base al combo di carte giocato	16
3	Sviluppo	18
3.1	Testing automatizzato	18
3.1.1	Morri Simone	18
3.1.2	Zangheri Marco	18
3.1.3	Azhar Hamail	19
3.2	Note di sviluppo	19
3.2.1	Morri Simone	19

3.2.2	Zangheri Marco	19
3.2.3	Azhar Hamail	20
4	Commenti finali	22
4.1	Autovalutazione e lavori futuri	22
4.1.1	Morri Simone	22
4.1.2	Zangheri Marco	22
4.1.3	Hamail Azhar	24
4.2	Difficoltà incontrate e commenti per i docenti	24
4.2.1	Zangheri Marco	24
A	Guida utente	25
B	Esercitazioni di laboratorio	27

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Il software vuole essere un mockup di risiko utilizzando come riferimento il manuale fornito da risiko.it¹ prendendo una regola da torneo, ovvero che il difensore si difende sempre col massimo delle sue forze, e ignorando per semplicità tutte le altre regole da torneo.

RisiKo ha vari elementi distinti fra di loro necessari per il funzionamento del gioco tra cui:

- Un tabellone di gioco dove sono rappresentati i vari territori.
- Truppe di vario colore per distinguere i giocatori e segnare il numero di armate in uno specifico territorio.
- Un mazzo di carte raffiguranti una figura e un territorio da utilizzare sia in fase iniziale per definire i territori posseduti ad inizio partita, sia come bonus per ottenere truppe bonus ad inizio del proprio turno.
- Delle carte che rappresentano vari obiettivi per determinare il vincitore della partita.
- Dadi per risolvere le azioni di gioco.
- Dalle 3 alle 8 persone per poter giocare se si seguono le regole classiche.

Questo ci porta ad avere dei giocatori che devono essere consapevoli delle loro potenziali azioni, una mappa che tiene traccia della disposizione di territori e truppe e un mazzo che fornisce carte ai giocatori.

¹<https://risiko.it/wp-content/uploads/2017/10/Regolamento-Risiko.pdf>

Nello specifico una partita comincia prendendo almeno 3 giocatori, gli si consegnano delle carte che determinano i territori iniziali che poi vengono rimescolate, dopo aver piazzato a turno tutte le truppe disponibili sul tabellone e aver preso la propria carta obiettivo comincia la partita con il primo giocatore scelto con un lancio di dado, a quel punto un turno si divide in rinforzo, attacco e movimento, si continua così finché un giocatore non raggiunge il suo obiettivo a fine del suo turno.

1.1.1 Requisiti funzionali

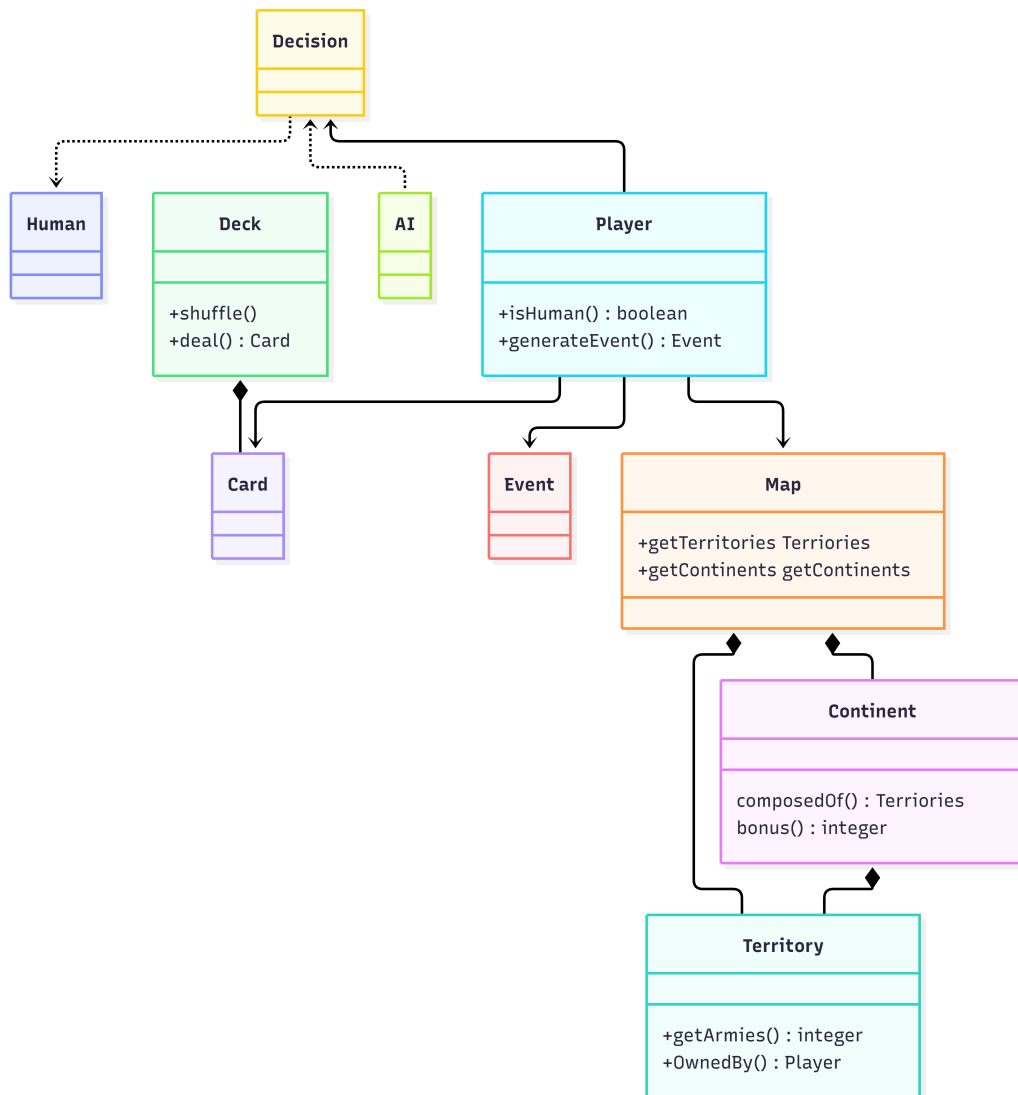
- Un gestore della partita per definire quali azioni sono possibili e differenziare le varie fasi di attacco, movimento e rinforzo.
- Un mazzo che può simulare le azioni di un mazzo reale (Mischiare le carte, dare le carte, etc...).
- Una mappa che tiene traccia delle armate presenti e mostra quali territori formano vari continenti.
- Bot che possono giocare in mancanza di giocatori umani.

1.2 Modello del Dominio

Durante una partita i giocatori possono consultare il tabellone che rappresenta la mappa di gioco contenente i vari continenti che a loro volta sono composti da territori che contengono le armate posizionate da ciascun giocatore. Un territorio si definisce in possesso di uno specifico giocatore se quest'ultimo ha delle armate nel territorio.

Ogni giocatore dispone di una propria mano, ovvero delle carte fornite dal mazzo composto da queste ultime.

I giocatori che possono essere umani o aventi un ai, entrambi generano degli eventi a seguito delle loro azioni, interagendo con la mappa.



Capitolo 2

Design

2.1 Architettura

Abbiamo usato il pattern MVC, con il codice diviso nei package `model`, `view` e `controller`.

La parte che ho fatto io è divisa così: mappa, dadi e combattimento stanno nel `model` e non fanno niente né della grafica né del controller; il disegno della mappa sta nella `view`, ma da fuori si usa solo tramite l'interfaccia `MapView`, che non contiene tipi di JavaFX; `MapClickHandler` sta nel controller, riceve i click dalla `view`, legge la mappa e non modifica niente. Per tenere aggiornato lo schermo uso il pattern Observer: `MapView` estende `GameObserver`, quindi riceve gli `Event` e la mappa si ridisegna da sola. Il flusso della partita, cioè la preparazione, i turni e l'esecuzione delle mosse, l'ho scritto nel `GameController`: prende le mosse dalle strategie dei giocatori, le esegue sul `model` e avvisa la `view` con gli eventi.

Per la mia parte cambiare libreria grafica vuol dire riscrivere solo `MapCanvas` e `DiceCanvas`: il `model` e `MapClickHandler` restano uguali.

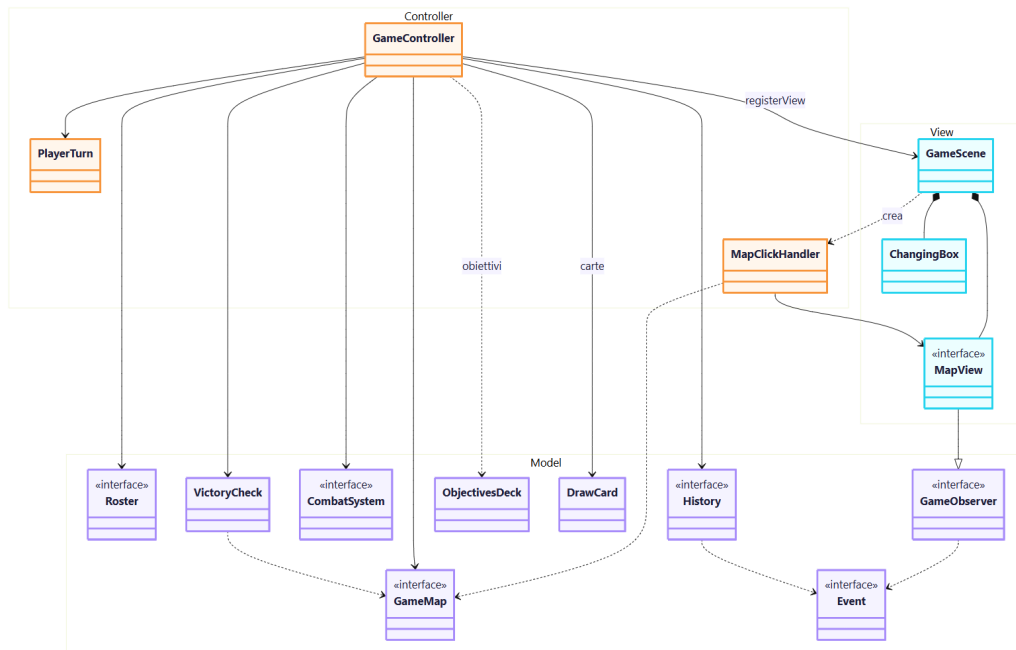


Figura 2.1: Architettura: i punti d'ingresso di view, controller e model

2.1.1 Carte e fasi del turno

La parte delle carte e delle fasi è divisa così: le carte, il mazzo e le fasi stanno tutti nel model. Nel controller, in particolare nella classe **PlayerTurn**, sono aggiunti metodi per informare i giocatori quando un giocatore ha vinto e quindi la partita è finita. Nella classe **GameScene** della view, c'è il metodo per far vedere ai giocatori, quale giocatore ha vinto. Inoltre, nella classe **CardView** viene implementata la view delle carte, cioè in base al tipo di carta, la classe cerca tra le risorse l'immagine carta di quel tipo e con quel nome e lo fa vedere al giocatore durante la Play Cards Phase. Nella classe **Deck** nel model, un mazzo viene popolato aggiungendo le carte ad una lista usando il metodo **add** delle liste. Per distribuire le carte, viene usata un'istanza random della classe **Random** che dato un numero a caso, prende la carta a quell'indice nella lista mazzo, la salva in una variabile **card** di tipo **Card**, la rimuove dalla lista (usando il metodo **remove**) e restituisce la carta a chiunque abbia chiamato il metodo. Per non generare un nuovo oggetto ogni volta, ho creato una field **random** da usare come random number generator. Per le varie fasi invece, prima controllo se il giocatore è umano o ai:

- se è ai, basta chiamare il metodo (implementato da Simone) per quella classe;

- se è umano, bisogna fare cast della startegy del player e poi chiamare il metodo.

Per l'ultima fase **MovePhase**, dopo che il giocatore ha spostato le armate, prima di passare al giocatore successivo (se ci sono) controllo e gestisco le eliminazioni dei giocatori e poi passo al giocatore successivo.

Per i giocatori abbiamo provato il più possibile a mantenerli separati dalla view in quanto sono componenti essenzialmente solo del model, nel far questo sono state necessarie varie classi che si distaccavano dalla implementazione dei giocatori e questi ultimi hanno vari metodi di "ingresso" per permettere al controller di ricevere le loro azioni, valide o meno. Tutto sommato sebbene non siamo riusciti ad avere una distinzione netta tra controller e view dove è possibile "staccarne e attaccarne" varie a uno stesso controller con molta facilità, data anche la complessità del progetto che ha richieste specifiche per la GUI oltre a richiedere tempi di progettazione ancora maggiori che non ci potevamo permettere, il risultato è per noi molto soddisfacente in quanto la view fa richieste solo al controller per ciò che gli serve e il model non sa nulla di come è stata progettata la view, e la view a sua volta non sa nulla di come è progettato il model.

Elementi positivi

- Si mostrano pochi, mirati schemi UML dai quali si deduce con chiarezza quali sono le parti principali del software e come interagiscono fra loro.
- Si mette in evidenza se e come il pattern architetturale model-view-controller è stato applicato, anche con l'uso di un UML che mostri le interfacce principali ed i rapporti fra loro.
- Si discute se sia semplice o meno, con l'architettura scelta, sostituire in blocco la view: in un MVC ben fatto, controller e modello non dovrebbero in alcun modo cambiare se si transitasse da una libreria grafica ad un'altra (ad esempio, da Swing a JavaFX, o viceversa).

2.2 Design dettagliato

2.2.1 Far produrre allo stesso tipo di entità risultati differenti. Morri Simone

Il primo e principale problema è : come far prendere decisioni o/e avere comportamenti diversi in base al tipo di giocatore, qui mi sono

imbattuto in una necessità di dare a ogni giocatore in campo il proprio comportamento, quindi lo Strategy Pattern è stato essenziale nella progettazione. Questo pattern mi permette di ridurre le implementazioni di player da quanti comportamenti diversi volevo a 1 dove espone ciò che un vero giocatore può fare, delegando le decisioni a PlayerStrategy dove ogni sua implementazione può fornire un risultato diverso, nel caso delle ai producono un risultato in base al contesto fornito e i giocatori umani tramite dei Builder interni possono costruire il risultato passo per passo. Facendo così non solo ho ridotto la quantità di codice duplicato ma è anche possibile aggiungerne diversi tipi di strategie ed eventualmente anche combinarne varie in uno stesso giocatore.

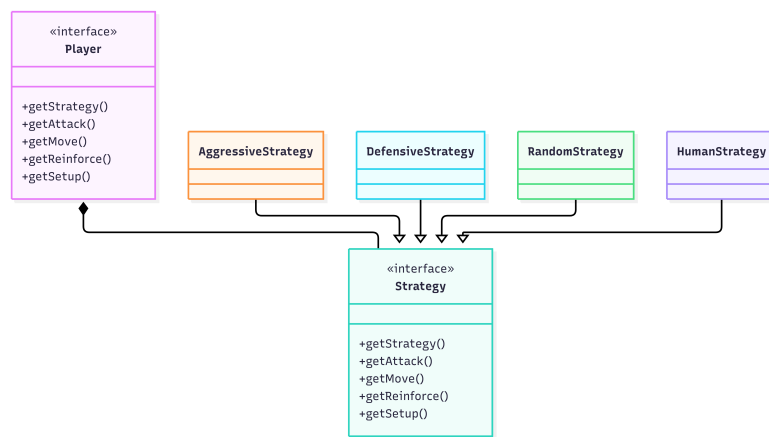


Figura 2.2: Strategy per i Player

2.2.2 Comunicazione per i Player. Morri Simone

Un altro problema è come comunicare le intenzioni dei giocatori al controller, quindi ho creato dei DTO nominandoli Event in modo da poter impacchettare tutto ciò che potrebbe servire ad un controller per eseguire l'azione, utili anche per poter creare uno storico della partita, fin da subito però sono sorti problemi in quanto esistono diversi tipi di eventi tutti con campi diversi, ciò rende impossibile utilizzarli senza fare un cast esplicito, ma anche qui non sappiamo la loro classe originale, quindi utilizzando il Visitor Pattern è possibile dare un'implementazione specifica a seconda di cosa voglio generare a partire da uno specifico Event, questa cosa è tornata utile nella History che è passata da un contenitore di eventi a un contenitore di Stringhe che rappresentano gli

eventi attraverso una implementazione specifica di Visitor. Utilizzando questo pattern inoltre permette anche ad altro programmatori di modificare gli eventi per ad esempio renderli persistenti oppure esportarli in altri formati per il salvataggio in ad esempio Json.

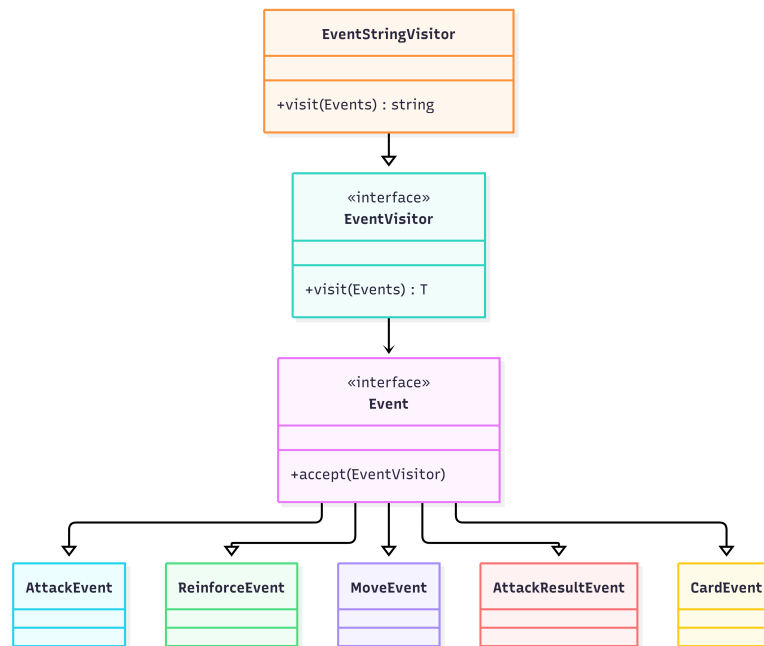


Figura 2.3: Modello per visitor

2.2.3 Creare i Giocatori. Morri Simone

Un ultimo problema non meno importante è come far costruire i vari giocatori che parteciperanno alla partita, visto che ognuno deve avere delle specifiche proprie. Qui utilizzando sempre dei DTO chiamati PlayerRequest posso gestire la creazione astruendo il giocatore dalla richiesta di creazione, e per rendere la creazione più pulita e facile da fare ho creato una Simple Factory che riceve le varie richieste dei giocatori e produce una lista di Player pronti da utilizzare create sotto le specifiche e per aderire all'utilizzo di ID proposto il tutto viene raccolto in Roster che alla creazione richiede tutte le richieste e permette di interrogare i giocatori chiedendo chi è il possessore di uno specifico ID o Colore,quest'ultimo astratto in RisikoColors.

2.2.4 Zangheri Marco. Costruire la mappa e collegarla ai giocatori

La mappa ha 42 territori, 6 continenti e 83 confini. Scriverla nel codice voleva dire centinaia di righe e ricompilare ogni volta che sbagliavo un confine, quindi l'ho messa in un file di testo, `world.txt`, che `MapLoader` legge riga per riga. `MapLoader` però non controlla niente: passa tutto a `GameMapBuilder`, e qui ho usato il pattern Builder. Il builder raccoglie continenti, territori e confini e solo alla fine, in `build()`, controlla che la mappa abbia senso: niente id doppi, niente confini verso territori che non esistono, nessun territorio isolato. Così una mappa sbagliata non si può creare: o esce giusta o ti dà errore.

Poi c'era da decidere come legare i territori ai giocatori. Avevamo valutato anche l'alternativa di far restituire a `Territory` direttamente il `Player` proprietario, che era più comoda, ma così la mappa dipendeva dai giocatori, che a loro volta usano già la mappa, e non si poteva più usare né testare da sola. Ho tenuto quindi solo l'id del proprietario, e anche i confini sono id. Per decidere ci siamo basati sulla meta-relazione, che tra gli errori di architettura mette proprio un modello che non si può riusare. Il prezzo è che l'id è una `String` e il compilatore non ti ferma se ne passi uno sbagliato. Per non riscrivere ogni volta la ricerca per id ho fatto `Registry<T extends Identifiable>`, che poi è stato usato anche per i giocatori.

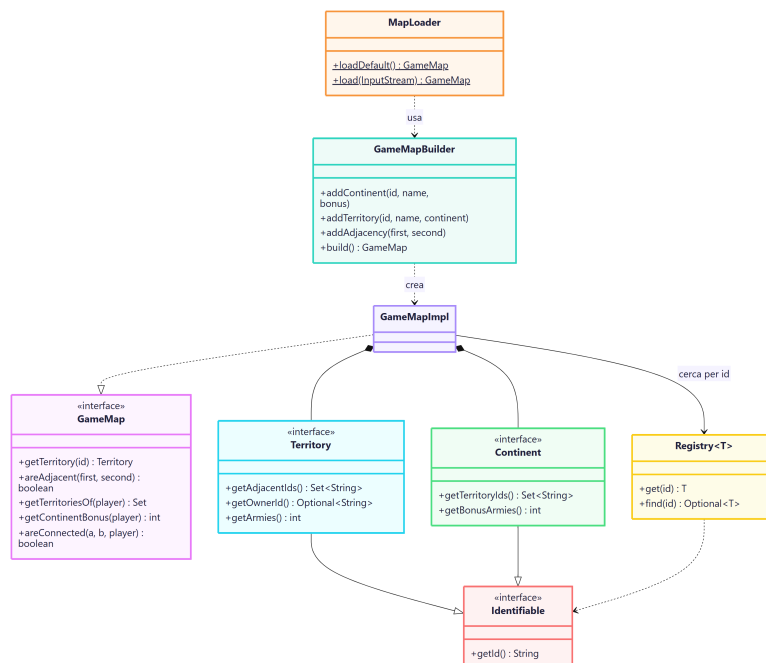


Figura 2.5: Costruzione della mappa

2.2.5 Zangheri Marco. Dadi sostituibili e combattimento

Per il combattimento il problema era testarlo: con i dadi veri il risultato cambia ogni volta e non puoi controllare se le perdite sono giuste. Quindi Dice l'ho fatto come interfaccia (pattern Strategy): in partita `CombatSystemImpl` usa `RandomDice`, nei test gli passo `FixedDice`, un dado finto a cui dico prima cosa deve uscire.

`resolve` tira i dadi, li confronta a coppie partendo dal più alto (in parità vince chi difende) e restituisce un `BattleResult` con i dadi usciti, le perdite di tutti e due e se il territorio è conquistato. All'inizio mi sembrava una complicazione che `resolve` non tocchi la mappa, e che le armate le debba togliere chi gestisce il turno, ma è proprio quello che lo rende facile da testare: c'è un test apposta che controlla che dopo uno scontro i territori siano rimasti uguali.

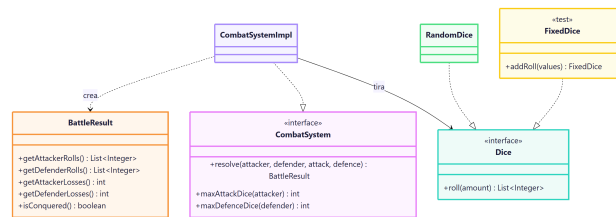


Figura 2.6: Dadi sostituibili e combattimento

2.2.6 Zangheri Marco. La mappa a schermo

La mappa si deve ridisegnare dopo ogni attacco o spostamento, ma chiamarla dal codice del turno voleva dire mettere la grafica dentro alla logica di gioco. Ho usato il pattern Observer: **GameObserver** ha un solo metodo, **onEvent**, la mappa lo implementa e si aggiorna da sola quando le arriva un evento. Per capire se un evento cambia il disegno uso un visitor, **RedrawIfNeeded** dentro **MapCanvas**: un attacco appena dichiarato no, il risultato di un attacco sì. Qui ho visto pro e contro del visitor: quando Simone ha aggiunto **CardEvent** la mia classe non compilava più finché non ho deciso cosa farne. Fastidioso, però me ne sono accorto subito e non giocando.

Il controller non vede mai **MapCanvas**, lo crea solo la schermata di gioco, e usa l'interfaccia **MapView**, dentro cui non c'è niente di JavaFX: i colori sono i **RisikoColors** del model e i territori sono id. Le posizioni stanno su una griglia fissa di 1000 per 600 e **ViewTransform** le riscalda sulla finestra; siccome disegno e click usano la stessa trasformazione, il territorio che clicchi è sempre quello che vedi.

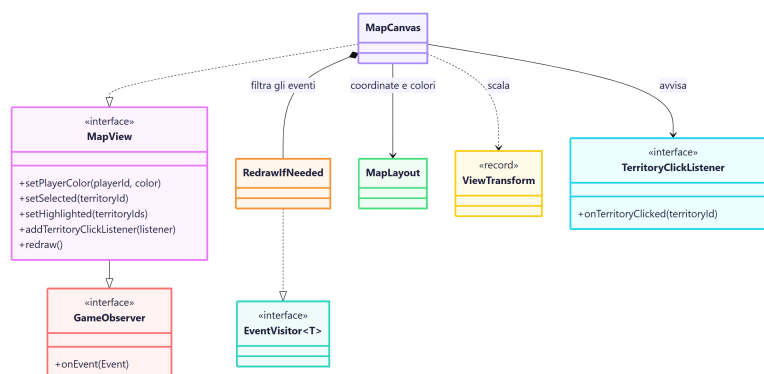


Figura 2.7: La mappa come osservatore degli eventi

2.2.7 Zangheri Marco. Cosa vuol dire un click

La mappa sa solo quale territorio hai cliccato, ma lo stesso click vuol dire cose diverse in base alla fase: in attacco il primo click è da dove attacchi e il secondo il bersaglio, nello spostamento il secondo è dove vai, mentre nel rinforzo e nella preparazione basta un click su un tuo territorio per metterci un'armata. Invece di mettere queste regole in **MapCanvas** ho fatto **MapClickHandler**, nel controller: col primo click scegli un tuo territorio con almeno due armate e la mappa evidenzia dove puoi andare, col secondo, se la destinazione va bene, avvisa chi ascolta e gli passa i due id. La mossa non la crea lui, quella resta alla parte dei giocatori: i due territori, insieme alle armate scelte con il contatore, li passo alla strategia del giocatore umano, che li raccoglie finché la mossa non è completa.

All'inizio c'era un metodo solo, **setTurn(giocatore, fase)**, perché pensavo che chi faceva il gestore dei turni avesse sempre tutte e due le cose insieme. Con **PlayerTurn** di Simone si è visto che non è così: la fase cambia più volte nello stesso turno e viene annunciata da sola, il giocatore cambia una volta per turno. L'ho quindi diviso in **setPlayer** e **setPhase**, e ho collegato **setPhase** ai cambi di fase. Poi ho collegato anche **setPlayer**: **PlayerTurn** annuncia chi gioca a ogni cambio di turno e alla fine della preparazione, con un avviso separato da quello delle fasi.

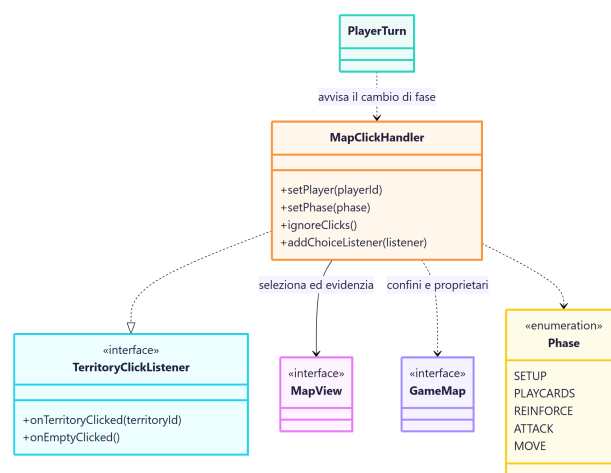


Figura 2.8: Gestione dei click

2.2.8 Zangheri Marco. Il flusso della partita

All'inizio il controller aveva solo i metodi pensati per il flusso della partita, ma vuoti: nessuno faceva partire il gioco, eseguiva le mosse o passava il turno. Su richiesta di Simone l'ho completato io. Adesso, quando si apre la schermata, il controller distribuisce gli obiettivi e fa la preparazione: ogni territorio ha già un'armata e, a turno, ognuno piazza le armate che gli restano, gli umani cliccando e i bot da soli. Poi partono i turni. Nel rinforzo il controller calcola le armate da piazzare, cioè un terzo dei territori, almeno tre, più il bonus dei continenti. In attacco, a ogni conferma prende la mossa dalla strategia del giocatore, la risolve con `CombatSystem`, toglie le perdite, gestisce la conquista e manda il risultato a storico, mappa e dadi con `publishBattle`; si può attaccare più volte, e alla fine lo spostamento passa il turno.

I bot giocano da soli con la loro strategia, ma ogni loro mossa la controllo prima di eseguirla, con le stesse regole del combattimento: così una mossa sbagliata viene ignorata invece di bloccare il gioco. Dopo ogni attacco controllo la vittoria, con `VictoryCheck` oppure se un giocatore ha conquistato tutta la mappa.

Il punto debole è che il controller è diventato grande: il flusso starebbe meglio in una classe sua, e il turno dei bot in un'altra.

2.2.9 Hamail Azhar. Costruire due tipi di mazzi

In un mazzo di carte territorio ci sono 42 carte di tipo territorio e altre 2 carte jolly mentre nel mazzo di carte obiettivo ci sono 14 carte obiettivo. Facendo prima questa distinzione, ho pensato di creare due classi `ObjectivesDeck` e `TerritoriesDeck`, per costruire i due mazzi, i metodi comuni a questi mazzi sono stati inseriti in una terza classe `Deck`. Nella classe `deck` ci sono metodi per popolare un mazzo di carte `populateTerritoryDeck` e `populateObjectiveDeck` a partire da classi enum (`Objective`, `CardTerritories` per aggiungere i territori alle carte territorio e `CardTroops` per aggiungere le truppe alle carte territorio). Inoltre, per aggiungere le carte jolly, che non hanno nessun territorio, ho usato un metodo `addJolly` che crea solo due carte jolly. Nella classe `TerritoriesDeck`, ci sono due metodi, uno per creare il mazzo con le carte territorio e uno per aggiungere le carte jolly. Entrambi però contengono un metodo `shuffle` che mischia le carte dopo che sono state inserite nel mazzo. Questi metodi vengono poi usati nella fasi di preparazione. Per risolvere il problema di ripopolare il

mazzo dopo che si è svuotato, ho aggiunto un controllo in `dealCard` che prima di distribuire una carta, controlla se il mazzo è vuoto e se lo è, lo ripopola da solo.

2.2.10 Hamail Azhar. Condizioni di vittoria

Per le carte, ho creato una classe comune `Card` che contiene costruttori per le varie carte, che possono essere carte territorio, carte jolly e carte obiettivo. Per controllare se un giocatore aveva raggiunto il suo obiettivo (assegnatogli durante la `SetupPhase`) ho prima creato due metodi `checkElimination` e `managePlayerElimination` dentro la `MovePhase`. Dopo che il giocatore ha spostato le sue truppe, è arrivato alla fine del suo turno, quindi con `checkElimination` controllo se qualche giocatore è stato eliminato e se è stato eliminato chiamo `managePlayerElimination` che controlla se eliminarlo era l'obiettivo di qualcuno. Questo metodo ritorna `true` solo se l'obiettivo di eliminare quel giocatore era del giocatore corrente, il che significa che il giocatore corrente ha vinto. Altrimenti, se l'obiettivo di eliminarlo era di un'altro giocatore, gli assegno un nuovo obiettivo (conquista 24 territori). Se poi `managePlayerElimination` non ha restituito `true`, chiamo `victoryCheck` per sapere se il giocatore corrente ha raggiunto il suo obiettivo. Dentro `victoryCheck`, prima controllo (tramite switch) qual'era l'obiettivo del giocatore corrente e poi nel case controllo se ha raggiunto quel obiettivo.

2.2.11 Hamail Azhar. Armate bonus in base al combo di carte giocato

La classe `CardBonus` serve per controllare se un giocatore ottiene delle armate bonus in base a un tris di carte che gioca. Il metodo principale è `bonusReinforcements` che prima calcola il numero di armate bonus in base al numero di territori occupati dal giocatore e poi chiama `calculateThreeBonus` per vedere se il combo di carte che il giocatore vuole giocare, gli assegnano delle armate in più. Per cui, prima calcola il numero di carte jolly, poi assegna truppe in base alla combinazione di carte giocata.

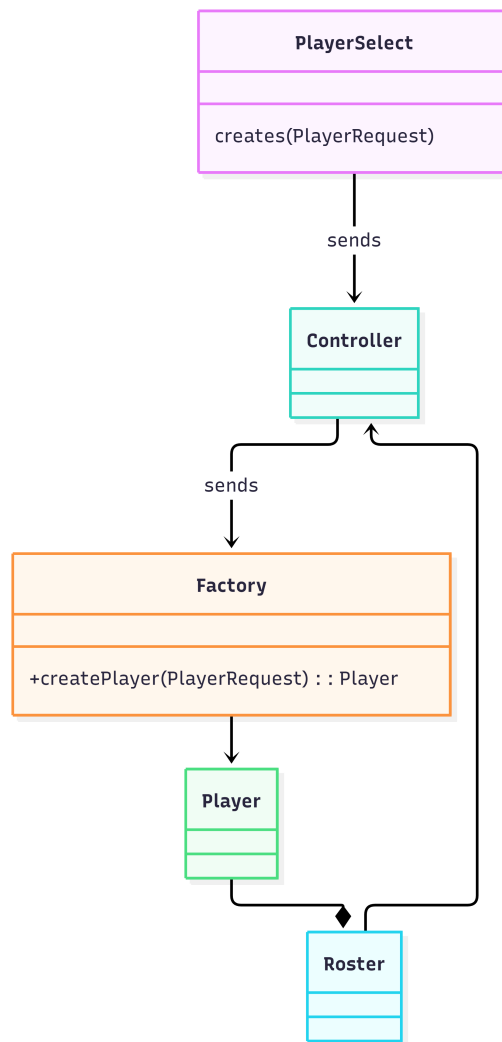


Figura 2.4: The player creation pipeline

Capitolo 3

Sviluppo

3.1 Testing automatizzato

3.1.1 Morri Simone

La parte di testing per i giocatori ho dovuto sottoporre a testing praticamente ogni mia implementazione delle strategy verificando che dessero l'output desiderato su una mappa fittizia e i vari builder di `HumanStrategy` producessero un output corretto, l'unico test che non è possibile considerare automatizzato è quello riguardante la `RandomStrategy` in quanto la sua implementazione sarebbe dovuta essere cambiata troppo, infatti anche fornendogli un seed gli output cambiano se vengono eseguiti tutti i test o no.

3.1.2 Zangheri Marco

Ho testato con JUnit 5 la parte di model: mappa, lettura del file, posizioni dei territori e combattimento, 14 test in tutto. Per mappa e combattimento uso mappe piccole fatte a mano, così ogni test prova una cosa sola e non dipende dal file, e i dadi li do io con `FixedDice`. `MapLoaderTest` e `MapLayoutTest` invece caricano la mappa vera e controllano che il file sia giusto: 42 territori, 6 continenti, confini che valgono nei due sensi e coordinate per ogni territorio, perché un territorio senza coordinate sarebbe in gioco ma non lo vedresti.

Per `MapCanvas`, `DiceCanvas` e `MapClickHandler` non ho fatto test automatici per mancanza di tempo. `MapClickHandler` si potrebbe testare con una `MapView` finta, visto che non usa JavaFX.

3.1.3 Azhar Hamail

- Ho testato che i metodi usati per le varie operazioni con il mazzo siano funzionali.
- Dato un set di carte che un giocatore volesse giocare, ho testato se ottengo il numero di rinforzamenti giusto.
- Infine, ho anche testato che il controllo delle condizioni di vittoria sia giusta.

3.2 Note di sviluppo

3.2.1 Morri Simone

- * Stream, Lambda e Optional utilizzo pervasivo ad esempio
- * JavaFx e le sue componenti ad esempio

Inoltre potrebbero essere presenti pattern di codice simile a quello altrui in quanto per imparare (principalmente JavaFx) ho seguito guide e discussioni sui forum su come utilizzare vari componenti che mi servivano e quindi il mio codice è venuto inconsciamente viziato da tecniche altrui, elencherei tutto ciò che ho utilizzato ma mi risulta impossibile in quanto non ne ho tenuto traccia vista la varietà. L'unico pezzo di codice copiato è questo dove appare più volte per risolvere lo stesso problema di ottenere combinazioni di un set di oggetti, appare come metodo di libreria di Apache-Commons, ma non volendo importare una intera libreria solo per uno snippet di codice lungo 3 righe la fonte più "affidabile" che ho trovato è probabilmente una slide di una lezione della Cornell, nello specifico la numero 16.

3.2.2 Zangheri Marco

- * Generici con parametro vincolato, `Registry<T extends Identifiable>` ad esempio
- * Stream, lambda, method reference e Optional per sommare i bonus dei continenti ad esempio
- * Ricerca in ampiezza (algoritmo classico di visita dei grafi, descritto per esempio in Cormen et al., *Introduction to Algorithms*) sui territori di un giocatore, per controllare lo spostamento ad esempio

- * Record per la trasformazione delle coordinate ad esempio
- * Lambda per gli eventi di JavaFX sul **Canvas** ad esempio
- * JavaFX per la parte grafica: il **Canvas** su cui disegno mappa e dadi, e il legame tra le sue dimensioni e quelle della finestra ad esempio
- * Varargs nei test, per passare le righe di un file finto direttamente come argomenti ad esempio
- * Switch con le frecce per smistare le righe del file della mappa ad esempio
- * **Registry** implementa **Iterable**, così territori e continenti si scorrono con un for-each ad esempio

Per alcune parti, segnalate qui sotto, ho preso spunto da guide e discussioni sui forum:

- * La formula che decide se il numero delle armate va scritto in bianco o in nero sopra il colore del giocatore qui: i pesi 0.299, 0.587 e 0.114 sono quelli della luminosità percepita dello standard ITU-R BT.601.
- * Il calcolo per adattare la mappa alla finestra senza deformarla e tenerla centrata qui: la scala è il minimo tra i due rapporti di larghezza e altezza, e i margini servono per centrarla.
- * Il modo per far seguire al **Canvas** la grandezza della finestra, visto che in JavaFX non si ridimensiona da solo: gli legghi larghezza e altezza a quelle del contenitore qui e lo ridisegni quando cambiano qui.
- * Il metodo dei test che trasforma delle righe di testo in un **InputStream**, così **MapLoader** e **MapLayout** leggono un file finto senza doverlo creare davvero qui: questo l'ho copiato così com'è da una discussione su un forum, e lo stesso metodo è anche in **MapLayoutTest**.

3.2.3 Azhar Hamail

- * Uso di funzionalità di JavaFX per le carte, ad esempio

Ho preso spunto da:

- * video youtube, come hudredvisionguy
- * spiegazioni di concetti come pattern observers e altri da geek-sforgeeks

- * per bug o altri errori incontrati durante l'implementazione, ho consultato vecchi post su **stackoverflow**
- * w3school per concetti base, ad esempio, se non mi ricordavo come usare hashmap
- * javanotes per la gestione delle classi deck e card

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Morri Simone

Ritengo che la mia parte sia in parte troppo ingegnerizzata sia che si mantenga in piedi grazie a pezze e fix, questo forse è dovuto al tempo mal gestito, a una poca comunicazione iniziale e alla mala gestione delle parti in comune. La mia parte la credo forte in quanto più punti sono stati gestiti per avere una quanto più facile integrazione nelle altre componenti. Una evidente debolezza è la troppa dipendenza dalle componenti dei miei compagni che non sono riuscito a risolvere anche a livello teorico.

4.1.2 Zangheri Marco

Mi sono occupato della mappa e dei continenti, dei dadi e del combattimento, del disegno della mappa e della gestione dei click, e verso la fine anche del flusso della partita nel controller. Siccome la mia parte è quella su cui poggiano le altre, ho cercato di chiudere presto le interfacce, tenerle piccole e non dipendere dalle classi degli altri: la mappa usa solo id e la grafica si usa solo tramite `MapView`. Secondo me è stata la scelta giusta, perché quando gli altri hanno collegato le loro parti non ho dovuto cambiare niente di importante.

Però ci sono cose che potevo fare meglio. La prima è il disegno: i territori sono cerchi collegati da linee e non le forme vere dei continenti, quindi si capisce chi confina con chi ma non sembra

un tabellone. In più `MapCanvas` è la mia classe più grande e fa un sacco di cose insieme, l'avrei dovuta dividere in due, una per i continenti e una per i territori.

La seconda sono gli id: usare le `String` ha tenuto separate mappa e giocatori, però il compilatore non distingue l'id di un territorio da quello di un giocatore, e se ne passi uno sbagliato te ne accorgi solo quando il programma gira. Col senno di poi avrei fatto due record piccoli, `TerritoryId` e `PlayerId`.

La terza è che la mappa si fida troppo di chi la usa: `GameMap` restituisce i territori veri, quindi chiunque può cambiare proprietario e armate senza passare da nessuna regola, e lo stesso vale per il combattimento, visto che `resolve` calcola e basta. Erano scelte per tenere le cose semplici e testabili, ma potevo dare i territori in sola lettura e mettere nella mappa dei metodi che fanno i controlli.

La quarta è il controller: con il flusso della partita è diventato grande e fa troppe cose insieme. Lo dividerei in una classe per il giro dei turni e una per i bot, lasciando al controller solo il compito di parlare con la schermata.

Queste cose le ho viste verso la fine, soprattutto per il tempo e per la poca comunicazione che c'è stata tra di noi all'inizio: alcune decisioni che riguardavano tutti e tre le abbiamo prese ognuno per conto suo.

Nella proposta iniziale mi ero preso anche il salvataggio della partita, che era fra le funzionalità opzionali, e alla fine non sono riuscito a farlo. Qui il problema è stato come ho gestito il tempo: ho scelto di dare la precedenza alle parti obbligatorie e ai test, e quando le ho chiuse mi sono accorto che non ne restava abbastanza per fare bene anche questa. Salvare una partita infatti non vuol dire salvare solo la mappa, ma tutto lo stato del gioco, e farlo di corsa avrebbe voluto dire consegnare una funzione a metà. Ho preferito consegnare meno cose, ma finite e testate. La strada comunque è aperta: la mappa si carica già da un file di testo, quindi per salvarla basterebbe riscrivere lo stesso formato con proprietari e armate aggiornati, e nel menu il pulsante per ricaricare una partita è già predisposto, solo disattivato.

Se si andasse avanti, la prima cosa che farei è una mappa a poligoni: le coordinate stanno già in un file separato dal model, quindi basterebbe estendere `MapLayout` per fargli leggere una lista di punti invece di un punto solo.

4.1.3 Hamail Azhar

La principale difficoltà che ho incontrato è che ho gestito male il tempo. All'inizio non mi erano ben chiare le funzionalità che dovevo implementare, cioè avevo un'idea generale ma le cose non le avevo strutturate/messe in ordine. Dopo un po' ho iniziato a distribuire ogni task in parti più piccole e gestirle una alla volta e questo mi ha reso il lavoro un po' più facile. Questo mi ha dato esperienza per gestire i progetti futuri.

4.2 Difficoltà incontrate e commenti per i docenti

4.2.1 Zangheri Marco

Scrivere il codice ha avuto le sue difficoltà, ma la più grossa è stata lavorare in tre su parti che si toccano. Ognuno portava avanti la sua, e le cose in comune, come gli eventi o il modo di identificare territori e giocatori, a volte cambiavano mentre qualcuno le stava già usando. Ho capito che quelle vanno decise tutti insieme all'inizio, prima di cominciare.

Dal lato opposto, le cose che mi porto dietro da questo progetto sono soprattutto due. La prima è Git: non l'avevo mai usato prima, e lavorando in tre ho imparato a usarlo sul serio, con i commit, i branch e le pull request su un repository condiviso. La seconda è quanto conta separare bene le parti di un programma. Tenendo la mappa e il combattimento senza niente di grafico, ho potuto testarli da soli, senza aprire nessuna finestra, e il controller non sa nemmeno che la mappa è disegnata con JavaFX. Quando ho dovuto cambiare il disegno, per esempio per la linea fra Alaska e Kamchatka, ho toccato solo la parte grafica e il resto è rimasto com'era.

Appendice A

Guida utente

Innanzitutto si aprirà con un menù principal ridotto all'osso, cliccando su new game ci porta a una selezione di giocatori dove:

- * è possibile scegliere il tipo di giocatore e il suo nome
- * il colore viene assegnato randomicamente, ma sempre dei soliti 6 classici si parla.
- * dopo aver creato 3 o più giocatori, di cui uno deve essere umano cliccando su start new game (che dovrebbe essere cliccabile) ci porta alla mappa di gioco.

una volta alla mappa di gioco avvengono tutte le fasi del risiko, la fase di setup avviene posizionando tutte le truppe in una sola mandata (per risparmiare tempo) e questo avviene per ogni giocatore. A seguire i giocatori si alternano i turni prima giocando le carte disponibili, poi rinforzando, poi attaccando e in fine muovendo le truppe. Ci sono varie variazioni dalle regole standard fatte per consentirci una implementazione più rapida, ad esempio dopo una conquista vengono spostate le armate vittoriose in automatico e per pareggiare i conti con i bot il giocatore può muovere una volta le sue truppe da un qualsiasi punto ad un altro nella sidebar di sinistra abbiamo i giocatori, di cui viene indicato il turno, sotto i risultati dell'ultimo combattimento, a seguire dei bottoni che cambiano in base alla fase, done fa passare la fase mentre se appare confirm serve per confermare l'azione che si vuole, oltre a selezionare il numero di truppe da utilizzare attraverso + e -. sotto si ha un Log della partita che riporta tutte le azioni sia dei bot e non scrollabile. In fondo il bottone show Objective mostra l'obiettivo del giocatore corrente che se completato porta alla vittoria. Quando è necessario cliccare sui territori basta sempli-

cemente cliccare sul pallino corrispondente ,del colore del proprietario, e se in combattimento o movimento cliccare due territori li prende come primo sorgente e secondo destinazione dell'azione che va confermata dopo aver selezionato un numero corretto di truppe, nel rinforzo invece un singolo clic aggiunge una truppa al territorio corrispondente: ATTENZIONE! non c'è un bottone di undo quindi cliccare consapevolmente.

Appendice B

Esercitazioni di laboratorio

Nessuno di noi vuole consegnare i laboratori